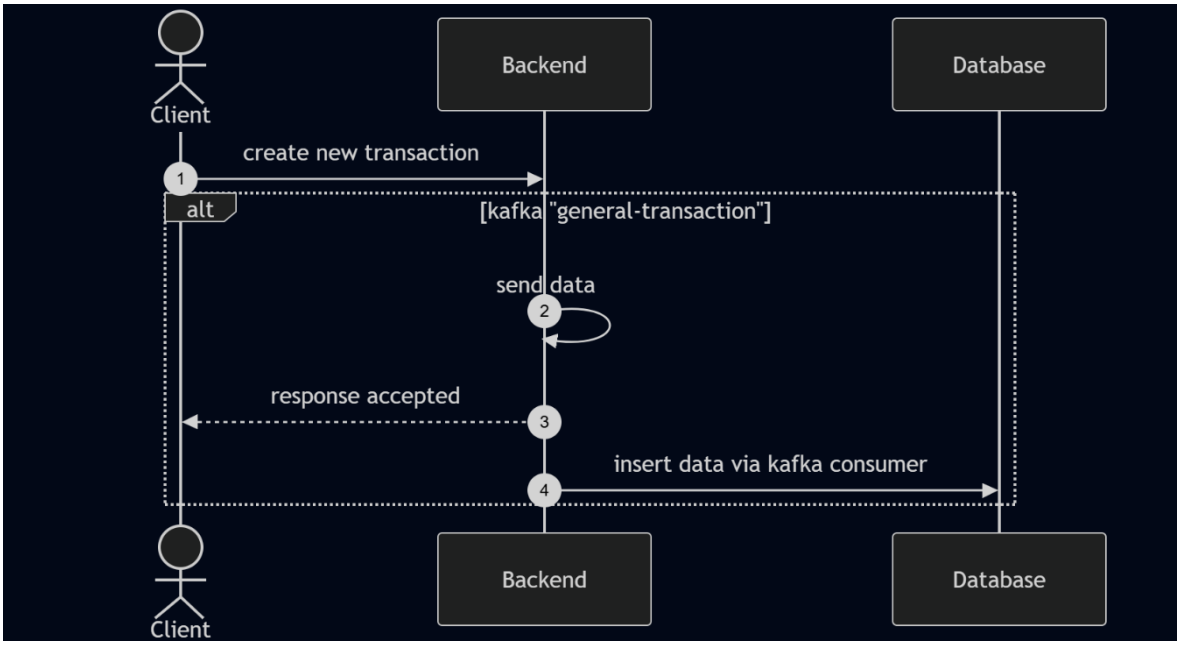
Transaction Processing System - Architecture Design

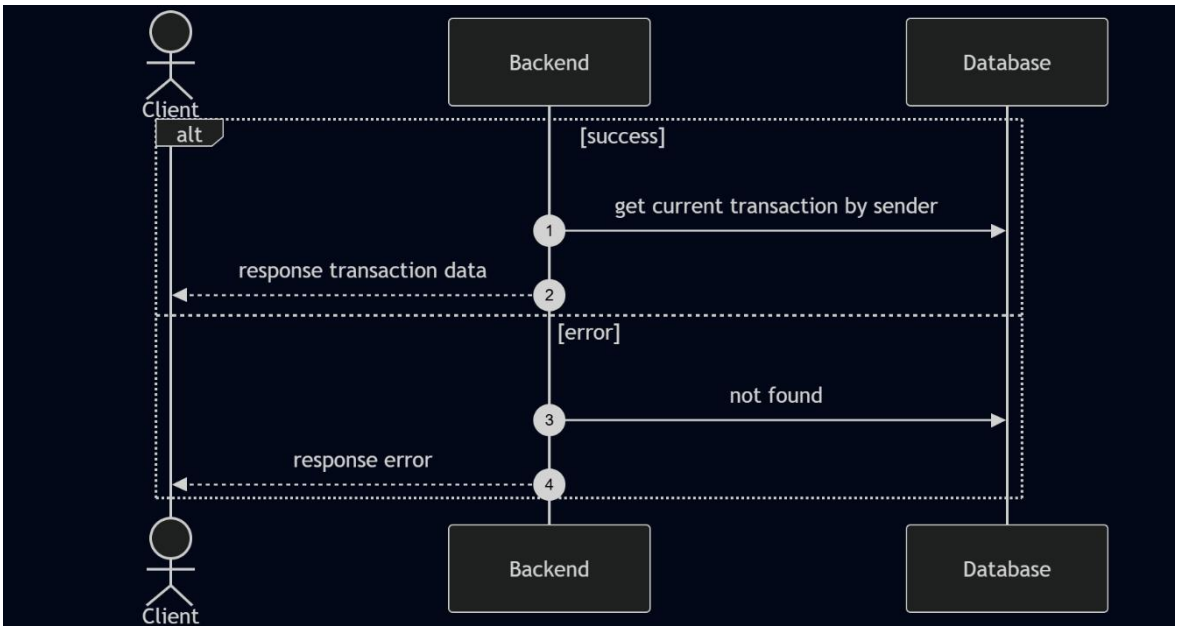
1. Architecture Overview

This system uses an event-driven approach to process transactions.
When a transaction request is received by the API, it is first validated and then published to Kafka. A worker service consumes the message and processes the transaction by updating balances and inserting ledger entries into PostgreSQL.

In this implementation, the API does not directly process the transaction. Instead, it delegates the work to a background worker through Kafka. This helps reduce load on the API and makes the system easier to scale.

High-level flow:





2. Components

The system consists of several main components:

- API Layer (Gin): receives requests, validates input, and publishes messages to Kafka
- Kafka: acts as a message broker to decouple API and processing
- Worker Service: consumes messages and processes transactions
- PostgreSQL: stores account balances and ledger entries
- Frontend (Next.js): displays balance and recent transactions using polling

3. Scalability

The system is designed to scale horizontally.
The API layer is lightweight and can be scaled independently since it only produces messages.
Kafka allows multiple consumers in the same consumer group, so workers can process messages in parallel.

If transaction volume increases, we can simply add more worker instances without changing the core logic.

Partitioning ensures that messages are distributed across multiple workers while maintaining order within a partition.

4. Idempotency

To prevent duplicate processing, each transaction uses a unique reference_id.

In the database, a unique constraint can be applied to ensure the same transaction is not inserted multiple times. If the same message is processed again, it will be ignored safely.

5. Reliability

Kafka ensures that messages are not lost because they are persisted until consumed.
If a worker crashes, the message will still be available and can be processed again. Database transactions are used to make sure balance updates are atomic.
In this implementation, a simple retry mechanism is also used in the worker to handle temporary failure

Retry mechanisms are implemented in the worker to handle transient failures. This ensures that temporary issues do not result in lost transactions.

6. Concurrency Handling

To prevent race conditions, row-level locking (SELECT FOR UPDATE) is used when updating account balances. This ensures that concurrent transactions do not corrupt the data and that

balances remain consistent.

7. Observability

Logging is implemented to track the flow of transactions from ingestion to processing. In a production environment, monitoring tools such as Prometheus and Grafana can be used to track system metrics and trigger alerts.

8. Trade-offs

The system introduces eventual consistency due to asynchronous processing. While this may delay balance updates slightly, it significantly improves scalability and system resilience.